Shared Brain Cell
User Manual

Operating Systems Final Project
Version 1.0

Group: Shared Brain Cells
John Rence Arenga, Kian Chris Po, Cj Sajonia,
Neil Sermeno, Kent Cedric Pacios

1. Introduction

Shared Brain Cell is a simple command-line operating system simulator. You type commands and the system responds. It lets you manage files, start and stop simulated processes, and control simulated memory. It does not control your real computer.

2. System Requirements

- Windows with Command Prompt (CMD)

- Python 3.x

- No outside libraries

3. Installation

1. Install Python 3 from python.org. Tick Add Python to PATH during setup.

2. Download or clone the project from GitHub.

3. Keep all five Python files in one folder: main.py, users.py, filesystem.py, processes.py, and memory.py.

4. How to Start Shared Brain Cell

1. Open the project folder in File Explorer.

2. Click the address bar, type cmd, and press Enter.

3. Type this command and press Enter:

```
python main.py
```

If Windows does not know python, type py main.py instead.

5. Login Instructions

The system asks for a username and a password. Your password stays hidden as you type. You get three tries.

Username	Password
admin	admin123
student	student123

After a good login you see the welcome banner and the prompt MiniOS>.

6. Command List

COMMAND: help

Purpose: Shows all available commands.

Syntax: help

Example: help

COMMAND: clear

Purpose: Clears the screen.

Syntax: clear

Example: clear

COMMAND: date

Purpose: Shows today's date.

Syntax: date

Example: date

COMMAND: time

Purpose: Shows the current time.

Syntax: time

Example: time

COMMAND: whoami

Purpose: Shows the user who is logged in.

Syntax: whoami

Example: whoami

COMMAND: systeminfo

Purpose: Shows simulated system details.

Syntax: systeminfo

Example: systeminfo

COMMAND: exit

Purpose: Shuts down the system.

Syntax: exit

Example: exit

COMMAND: dir

Purpose: Lists the files and folders in the current folder.

Syntax: dir

Example: dir

COMMAND: mkdir

Purpose: Creates a new directory.

Syntax: mkdir <folder>

Example: mkdir Documents

COMMAND: create

Purpose: Creates an empty file.

Syntax: create <file>

Example: create notes.txt

COMMAND: write

Purpose: Saves text in a file.

Syntax: write <file> <text>

Example: write notes.txt hello world

COMMAND: read

Purpose: Shows the content of a file.

Syntax: read <file>

Example: read notes.txt

COMMAND: delete

Purpose: Deletes a file or an empty folder.

Syntax: delete <name>

Example: delete notes.txt

COMMAND: cd

Purpose: Opens a folder. Use cd .. to go back.

Syntax: cd <folder>

Example: cd Documents

COMMAND: process

Purpose: Lists all running processes with PID, name, and status.

Syntax: process

Example: process

COMMAND: run

Purpose: Starts a simulated process and gives it a PID.

Syntax: run <name>

Example: run calculator

COMMAND: kill

Purpose: Ends a process by its PID.

Syntax: kill <pid>

Example: kill 1

COMMAND: memory

Purpose: Shows total, used, and free memory.

Syntax: memory

Example: memory

COMMAND: allocate

Purpose: Uses an amount of memory in MB.

Syntax: allocate <MB>

Example: allocate 200

COMMAND: free

Purpose: Releases an amount of memory in MB.

Syntax: free <MB>

Example: free 200

7. Common Errors

What you type	What you see	Why
hellooo	Unknown command.	The command does not exist. Type help.
read missing.txt	Error: File does not exist.	The file is not in the current folder.
kill 999	Error: Process does not exist.	No process has this PID.
allocate 5000	Error: Not enough free memory.	You asked for more than the free memory.
free 5000	Error: Cannot free more than the used memory.	You asked to release more than is used.
mkdir Docs (twice)	Error: Name already exists.	A file or folder has this name.

kill abc	Error: Please enter a whole number.	The PID must be a number.
mkdir	Error: Missing argument.	The command needs a name.

8. How to Exit Shared Brain Cell

Type exit at the prompt and press Enter. The system shows the shutdown message and returns you to CMD.

```
MiniOS> exit
Shutting down MiniOS...

Goodbye!
```

Note: files, processes, and memory changes are not saved. They reset every time you start the program.